

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 6

Operating Systems

Inside the xv6 File System

FAF-241, Ciur Alexandrina

Checked by:
Reitman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

1 Introducere

Scopul laboratorului este înțelegerea sistemului de fișiere din xv6, care conține aceleași idei ca ext4 (inode-uri, directoare ca tabele, hard link-uri, jurnal) într-un cod mic și ușor de citit. Lucrarea are patru părți:

1. citirea informațiilor afișate de `ls` și decodarea tipurilor din `kernel/stat.h`;
2. scrierea programului `mystat` și observarea hard link-urilor;
3. măsurarea limitelor definite în `kernel/fs.h`;
4. scrierea programului `find`, care parcurge recursiv arborele de directoare.

Mediu de lucru: Ubuntu 26.04.1 LTS, compilator `riscv64-linux-gnu-gcc`, QEMU 10.2, arborele `~/xv6-riscv`. Fiecare program nou a fost adăugat în `UPROGS` din `Makefile`.

2 Partea 1. Ce arată xv6

După pornirea sistemului cu `make clean ; make qemu` am rulat:

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
...
mystat         2 25 36320
console        3 26 0
$ mkdir a
$ ls a
.              1 27 32
..             1 1 1024
```

Fiecare linie conține: nume, tip, număr de inode, dimensiune în octeți. Conținutul relevant din fișierele sursă:

```
$ cat kernel/stat.h
#define T_DIR    1 // Directory
#define T_FILE   2 // File
#define T_DEVICE 3 // Device
...
$ grep -n "DIRSIZ" kernel/fs.h
54:#define DIRSIZ 14
$ grep -n -B1 -A4 "struct dirent" kernel/fs.h
58:struct dirent {
59-  ushort inum;
60-  char name[DIRSIZ] __attribute__((nonstring));
61-};
```

Observe

(1) Ce înseamnă tipurile 1, 2 și 3? Conform `kernel/stat.h`: `T_DIR` = 1 (director), `T_FILE` = 2 (fișier obișnuit), `T_DEVICE` = 3 (dispozitiv; în listă, `console`).

(2) Ce inode are directorul rădăcină și de ce `.` și `..` coincid? Rădăcina are inode-ul **1**. Ea nu are director părinte, deci intrarea `..` din rădăcină pointează tot spre ea însăși; de aceea `.` și `..` apar ambele cu inode 1 și dimensiunea 1024.

(3) Cât de mare este o intrare de director și ce conține directorul gol `a`? O intrare (`struct dirent`) are un `ushort inum` (2 octeți) și `char name[DIRSIZ]` cu `DIRSIZ` = 14, deci $2 + 14 = 16$ octeți. Directorul gol `a` are dimensiunea 32 de octeți, adică $32/16 = 2$ intrări: `.` (inode 27, adică el însuși) și `..` (inode 1, rădăcina).

3 Partea 2. Programul `mystat`

`mystat` apelează `stat()` pentru fiecare argument și afișează tipul, inode-ul, numărul de legături și dimensiunea.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
  if(t == T_DIR)
    return "directory";
  if(t == T_FILE)
    return "file";
  if(t == T_DEVICE)
    return "device";
  return "?";
}

int
main(int argc, char *argv[])
{
  struct stat st;
  for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }
  exit(0);
}
```

Notă: `st.size` este `uint64`, iar `printf`-ul din xv6 nu are `%l`; fără cast la `int`, compilarea cu `-Werror` eșuează. Linia adăugată în `Makefile`: `\t$U/_mystat\`.

Rulare pe disc curat:

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 2, size 1024 bytes
console: device, inode 26, links 1, size 0 bytes
```

Diferențele față de exemplul din enunț (`console` are inode 26 în loc de 25, iar `.` are `links 2` în loc de 1) se explică prin faptul că `mystat` ocupă un inode în plus, iar `mkdir a` adaugă legătura `a/..` către rădăcină.

Experimentul cu hard link-uri:

```
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$ rm README
```

```
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
$ cat r2 | wc
48 336 2441
```

Liniile relevante din cod:

```
$ grep -n "nlink" kernel/sysfile.c
...
239:    dp->nlink--;
244:  ip->nlink--;
...
$ grep -n -A25 "^iput" kernel/fs.c
357:  int last = (ip->ref == 1 && ip->valid && ip->nlink == 0);
...
360:  if (last) {
...
365:    itrunc(ip); // free the data blocks (type stays nonzero on disk)
```

```
$ grep -n -B2 -A12 "struct dinode" kernel/fs.h
32:struct dinode {
33-  short type;              // File type
34-  short major;             // Major device number (T_DEVICE only)
35-  short minor;             // Minor device number (T_DEVICE only)
36-  short nlink;             // Number of links to inode in file system
37-  uint size;               // Size of file (bytes)
38-  uint addrs[NDIRECT + 1]; // Data block addresses
39-};
```

Observe

(1) Ce s-a schimbat și ce nu după `ln` și `rm`? După `ln README r2`, ambele nume au **același inode (2)** și `links 2`: sunt două nume ale aceluiași fișier, nu două copii. După `rm README`, `r2` are tot inode 2 și aceeași dimensiune (2441 octeți), dar `links 1`; `cat r2 | wc` dă `48 336 2441`, deci conținutul este intact. S-a schimbat doar contorul de legături și dispariția unui nume din director, nu și datele sau inode-ul.

(2) Unde se decrementează contorul și când se eliberează blocurile? În `sys_unlink`, linia **244** din `kernel/sysfile.c` (`ip->nlink-;`). Linia 239 (`dp->nlink-`) se referă la directorul părinte, când se șterge un subdirector (dispare legătura `..`). Blocurile de date **nu** se eliberează în `unlink`, ci în `iput`: când `ip->ref == 1` și `ip->nlink == 0` (ultima referință la un inode fără legături), se apelează `itrunc(ip)`, care eliberează blocurile.

(3) Ce câmpuri are un inode xv6? `struct dinode` conține: `type`, `major` și `minor` (doar pentru dispozitive), `nlink`, `size` și `addrs[NDIRECT+1]` (adresele blocurilor de date). Nu există câmpuri pentru permisiuni, proprietar sau timpi, și nici un tip pentru legături simbolice.

4 Partea 3. Măsurarea limitelor

```
$ grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ" kernel/fs.h
5:#define BSIZE   1024 // block size
26:#define NDIRECT   12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
$ make clean ; make fs.img 2>&1 | grep nmeta
```

```
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
```

Test cu un nume lung:

```
$ mkdir averyveryverylongname
$ ls
...
console        3 26 0
averyveryveryl 1 27 32
```

Observe

(1) Ce arată `ls` pentru `averyveryverylongname`? Arată `averyveryveryl`, adică numele este **tăiat la 14 caractere**. Constanta care explică acest lucru este `DIRSIZ` = 14 (câmpul `name[DIRSIZ]` din `struct dirent`).

(2) Dimensiunea maximă a unui fișier. Un bloc indirect conține `NINDIRECT` = `BSIZE`/4 = 1024/4 = 256 numere de bloc. Numărul maxim de blocuri este 12 + 256 = 268, deci:

$$(12 + 256) \times 1024 = 268 \times 1024 = 274\,432 \text{ octeți } (= 268 \text{ KiB}).$$

(3) Structura discului. Din linia `nmeta 47 ... blocks 1953 total 2000`:

Zonă	Blocuri
boot	1
superblock	1
log	31
inode-uri	13
bitmap	1
metadate (nmeta)	**47**
date	**1953**
total disc	**2000**

Verificare: $1 + 1 + 31 + 13 + 1 = 47$ și $47 + 1953 = 2000$. Un `struct dinode` are 64 de octeți ($4 \times 2 + 4 + 13 \times 4$), deci un bloc conține 16 inode-uri, iar cele 13 blocuri de inode-uri permit $13 \times 16 = 208$ de inode-uri.

5 Partea 4. Programul `find`

`find <dir> <n>` afișează fiecare cale din `<dir>` al cărei ultim component este `<n>`. Implementarea pornește de la `user/ls.c`: deschide directorul, citește intrările `struct dirent`, ignoră intrările libere (`inum == 0`), nu intră în `.` și `..`, compară numele cu `strcmp`, apelează recursiv pe subdirectoare și închide fiecare descriptor înainte de întoarcere (limita este 16 fișiere deschise per proces). Argumentul opțional `-n` (stretch) afișează și numărul de inode.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "kernel/fs.h"
#include "user/user.h"

void
find(char *path, char *name, int showino)
{
  char buf[512], *p;
```

```
  int fd;
  struct dirent de;
  struct stat st;

  if((fd = open(path, 0)) < 0){
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }
  if(fstat(fd, &st) < 0){
    fprintf(2, "find: cannot stat %s\n", path);
    close(fd);
    return;
  }
  if(st.type != T_DIR){
    fprintf(2, "find: %s is not a directory\n", path);
    close(fd);
    return;
  }
  if(strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)){
    fprintf(2, "find: path too long\n");
    close(fd);
    return;
  }

  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';
  while(read(fd, &de, sizeof(de)) == sizeof(de)){
    if(de.inum == 0)
      continue;
    memmove(p, de.name, DIRSIZ);
    p[DIRSIZ] = 0;
    if(strcmp(p, ".") == 0 || strcmp(p, "..") == 0)
      continue;
    if(stat(buf, &st) < 0){
      fprintf(2, "find: cannot stat %s\n", buf);
      continue;
    }
    if(strcmp(p, name) == 0){
      if(showino)
        printf("%s %d\n", buf, st.ino);
      else
        printf("%s\n", buf);
    }
    if(st.type == T_DIR)
      find(buf, name, showino);
  }
  close(fd);
}

int
main(int argc, char *argv[])
{
  if(argc < 3 || argc > 4){
    fprintf(2, "usage: find <dir> <name> [-n]\n");
    exit(1);
  }
  int showino = (argc == 4 && strcmp(argv[3], "-n") == 0);
  find(argv[1], argv[2], showino);
  exit(0);
}
```

Modificarea din `Makefile` (ultimele linii din `UPROGS`):

```
        $U/_sync\
        $U/_mystat\
        $U/_find
```

Test pe disc curat (`make clean ; make qemu`):

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
```

Stretch: argumentul `-n`

```
$ find . r2 -n
./a/b/r2 30
./a/r2 31
./r2 3
$ mystat cat
cat: file, inode 3, links 2, size 36728 bytes
```

Dintre cele trei fișiere `r2`, doar `./r2` (inode 3) este **același fișier** cu `cat`: este un hard link creat cu `ln cat r2`, iar `cat` are `links 2`. Fișierele `a/b/r2` (inode 30) și `a/r2` (inode 31) sunt fișiere distincte care doar au același nume.

6 Concluzii

Am urmărit în practică modul în care xv6 organizează datele pe disc. Un director este un fișier obișnuit ce conține o secvență de intrări de 16 octeți (inode + nume de maximum 14 caractere), iar numele sunt doar etichete către inode-uri; de aceea `ln` creează un al doilea nume pentru același fișier, iar `rm` doar scade contorul `nlink`. Blocurile de date sunt eliberate abia în `iput`, când nu mai există nici nume, nici referințe deschise. Limitele sunt direct vizibile în constantele din `kernel/fs.h`: nume de 14 caractere, un fișier de cel mult 274 432 octeți (12 blocuri directe și unul indirect) și un disc de 2000 de blocuri, din care 1953 de date. Programele `mystat` și `find` au confirmat aceste idei: `mystat` a arătat inode-urile și contoarele de legături, iar `find -n` a demonstrat că două căi cu același nume pot fi fișiere diferite, în timp ce două căi cu nume diferite pot fi același fișier.